Glitch-Free Clock Multiplexer with Active Decision Logic, Symmetric Failover and Safe Recovery for Multi-Clock SoC Systems

Thuan Van Le*, Hai Minh Truong†
*Phenikaa School of Computing, Phenikaa University
†Faculty of Electrical and Electronic Engineering, Phenikaa School of Engineering, Phenikaa University

***Abstract*—Dynamic and reliable switching among asynchronous clock sources is a persistent hurdle in advanced SoC and FPGA systems, where conventional clock multiplexers frequently suffer from glitches, metastability, or deadlock, especially when a clock source unexpectedly halts. This work introduces a glitch-free clock multiplexer architecture featuring autonomous and symmetric failover, permission-driven selection logic, and integrated activity sensing across clock domains. By combining cross-domain activity detection, a permission-based arbitration scheme, and a two-stage synchronizer with an augmented edge-capture mechanism, the design ensures clean, deterministic handover and robust system recovery without requiring an external reference clock. Asynchronous force-disable paths further guarantee safe transitions under all failure scenarios. Comprehensive simulation and FPGA prototyping on Intel Cyclone V validate the architecture's effectiveness across normal operations, clock failures, and recovery events, with resource consumption as low as 12 ALMs and up to 496.52 MHz operating frequency. Compared to timer-based and cross-coupled approaches, the proposed solution demonstrably enhances resiliency and determinism while maintaining compact hardware footprint.**

***Index Terms*—Clock multiplexer, glitch-free switching, asynchronous clocks, failover, deadlock, metastability, FPGA, System-on-Chip (SoC), clock domain crossing, hardware reliability, fault tolerance, activity detection, synchronizer, permission-based arbitration**

I. Introduction

As digital systems grow in complexity and demand greater adaptability, reliable clock switching becomes a critical concern, especially in applications such as system-on-chip (SoC) platforms and FPGA-based designs. Clock multiplexers, used to select between multiple clock sources for dynamic frequency scaling, power optimization, or fault recovery, must guaranty robust and predictable operation to maintain system integrity.

However, ensuring seamless and safe transitions between asynchronous clock sources is non-trivial. Conventional multiplexing methods are often vulnerable to hazardous phenomena such as output glitches (unwanted pulses generated during select transitions) and metastability (unpredictable logic states arising from asynchronous control signal crossings). These effects can compromise downstream logic, induce timing violations, and ultimately threaten system reliability.

A range of prior architectures have attempted to address these issues through techniques such as break-before-make switching and watchdog-driven recovery. Yet, limitations remain: many solutions either compromise between glitch immunity and reliable failover, or introduce significant implementation overhead, manual intervention requirements, or susceptibility to deadlock and incomplete recovery.

Motivated by these challenges, this work presents a novel glitch-free clock multiplexer architecture capable of autonomous and safe switchover between asynchronous clock domains, even in the presence of faults or clock halts. Our design achieves robust functional safety and high speed with only modest resource overhead, as validated through FPGA implementation and detailed experimental comparisons.

The rest of this paper is organized as follows: Section II reviews the limitations of conventional and state-of-the-art multiplexer designs. Section III introduces the proposed architecture and its operation. Section IV presents functional verification and hardware evaluation, and Section V concludes the work.

II. Related Work

A. Challenges in Traditional Clock Multiplexer Design

Conventional clock multiplexer designs, such as those based on combinational AND/OR logic, are prone to glitches and metastability. As shown in Fig. 1, changes in the select `sel` input during active clock periods can lead to output glitches which are short, unwanted pulses violating set-up and hold times, and potentially propagating errors through sequential logic.

Additionally, asynchronous crossings without robust synchronization may induce metastability, especially when control signals like `sel` toggle near clock edges and are not protected by multi-stage synchronizers.

B. Break-Before-Make and Timer-Based Approaches

The break-before-make (BBM) technique, often implemented using cross-coupled flip-flops, was introduced to address the fundamental shortcomings of simpler combinational approaches, which are vulnerable to both metastability and glitches during clock transitions. By ensuring that only one clock enable is ever active during handover, BBM architectures effectively eliminate the risk of metastability and thus provide a substantial improvement in operational robustness.

However, despite its simplicity and effectiveness in resolving metastability, BBM does not fully address the glitch problem in all failure scenarios. In particular, if the selected clock

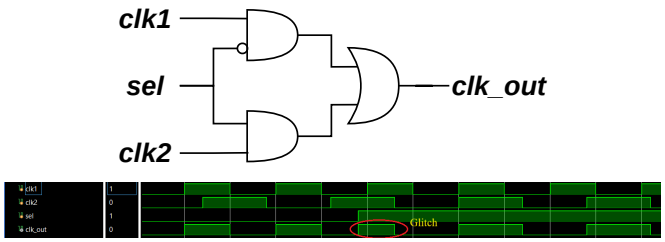

Fig. 1. Traditional AND/OR-based clock multiplexer: logic structure (top); waveform demonstrating output glitch caused by select changes during active clocks (bottom).

halts while remaining in a high state, the output `clk_out` can become locked at logic high, failing to switch to a viable clock source and resulting in a deadlock condition. This vulnerability is illustrated in the lower part of Fig. 2, where the output remains stuck until the faulted clock resumes toggling. Thus, while BBM markedly advances reliability compared to earlier designs, it remains susceptible to output glitches and deadlock in the presence of certain clock failures, motivating the need for an even more robust solution.

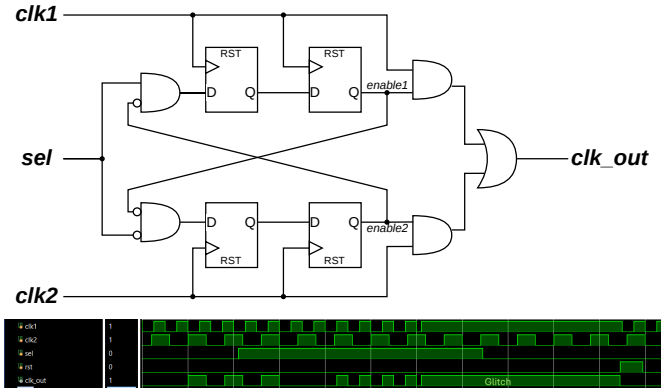

Fig. 2. Break-before-make multiplexer: cross-coupled flip-flop schematic (top); deadlock when a selected clock halts while high (bottom).

Timer-based (watchdog) clock multiplexers represent a significant improvement over BBM solutions through their use of dedicated timers to detect stalled clock domains, as illustrated in Fig. 3. When inactivity is detected, these circuits force the output to a defined safe state (typically logic Low) by asserting a reset, effectively eliminating the risk of deadlock when a clock halts high. In particular, if the selected clock resumes toggling after a fault, the timer-based approach can automatically recover the output operation that `clk_out` will follow the recovered clock without further user intervention.

However, the principal limitation of timer-based designs lies in their *passive handover policy*. If the selected clock fails and the selector signal (`sel`) remains unchanged, the output is merely forced to the safe state and does not transfer to an alternate live clock source. Only by either toggling the selector or waiting for the original clock to recover does normal operation resume. This lack of autonomous, immediate switchover to any available healthy clock restricts the system's applicability for high-availability or fail-operational scenarios, where uninterrupted output is essential.

Furthermore, timer-based implementations incur a performance penalty, as their frequency of operation (maximum Fmax) is typically limited by the additional logic and constraints introduced by the watchdog circuits. As shown in Table I, our proposed GFCM not only overcomes the passive handover obstacle, but also outperforms the timer-based approach in terms of maximum achievable frequency and overall resource efficiency.

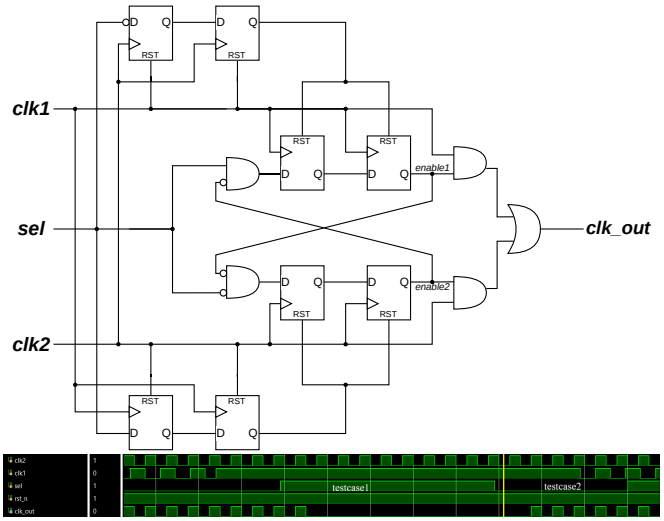

Fig. 3. Timer-based multiplexer: schematic with timers per clock source (top); waveform showing recovery upon selector change after clock failure (bottom).

C. Motivation for This Work

The prior architectures each tackle select failure modes: combinational designs offer simplicity, break-before-make (BBM) methods suppress glitches and metastability, and timer-based schemes add forced-safe outputs and automatic recovery. However, none alone guaranties glitch immunity, deadlock freedom, and fully autonomous, immediate switchover to any healthy clock source.

Our architecture intentionally integrates these advantages, while overcoming the remaining limitations, enabling robust failure detection, rapid and automatic switchover between clocks, and high-frequency operation within a unified framework.

III. Proposed Architecture

This section presents the detailed architecture of the proposed glitch-free clock multiplexer. The system is composed of three functionally distinct but closely interconnected layers, as illustrated in Fig. 4. This layered approach enables

rapid and robust failover in response to clock source failure, as well as reliable, glitch-free operation during clock transitions. While the architecture is designed to strongly favor switching to a functioning clock whenever possible, some rare or pathological failure scenarios may temporarily prevent immediate switchover; however, the system consistently seeks to activate any available live clock whenever feasible.

A. System Overview

Figure 4 displays the high-level block diagram of the proposed clock multiplexer. The circuit accepts two asynchronous clock inputs `clk1`, `clk2` and a user selector `sel`, and outputs a single glitch-free clock `clk_out`. Internally, the design is decomposed into three sequential processing layers, each with clearly delineated responsibilities:

- **Layer 1: Activity Sensor** : Continuously monitors the activity of each input clock by sampling it using the other clock's domain. For each clock, a real-time health signal is generated that indicates whether that specific clock is considered failed, as observed by the opposite clock domain.
- **Layer 2: Decision Making** : Receives the individual clock health signals from Layer 1 along with the user preference, and uses this information to dynamically select which clock should be considered for output. This layer generates "desired clock" signals, which indicate the preferred source based on current operating conditions but do not directly control the output gating logic.
- **Layer 3: Safe Handover** : Converts the "desired clock" signals from Layer 2 into the final output enables, guaranteeing that only one clock is active at the output at any time. It implements synchronized enable registers and mutual exclusion logic to ensure all switching is glitch-free, honoring the break-before-make principle during clock transitions.

This modular decomposition, as visually depicted in Fig. 4, clarifies both the dataflow and the control dependencies of the system, and forms the basis for ease of analysis and future extensibility.

B. Layer 1: Activity Sensor

The bottom layer, **Activity Sensor**, interfaces directly with the two asynchronous clock domains. Utilizing cross-domain sampling, each clock is sampled in the other's domain, and transitions are detected by XORing consecutive samples[1]. A saturating synchronous counter, running in the opposing clock domain, accumulates the number of cycles since the last detected transition. If this counter exceeds a predefined threshold (set here to 3), a failure flag (`fail1` or `fail2`) is asserted for that clock. This approach robustly distinguishes between brief jitter and true clock failure, and ensures fast detection without the need for a reference clock. The outputs of this layer, `fail1` and `fail2`, abstractly represent the health status of `clk1` and `clk2`, and are passed forward in the block diagram as shown.

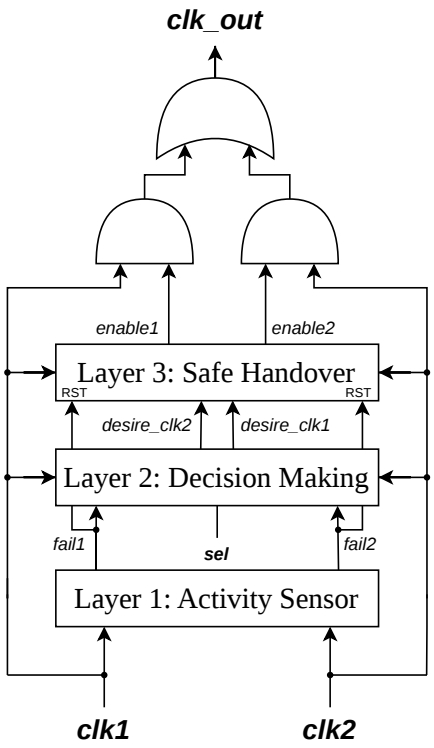

Fig. 4. Block diagram of the proposed clock multiplexer, illustrating the clear separation of monitoring, decision, and handover logic across three layers.

C. Layer 2: Decision Making

Layer 2 is responsible for real-time assessment and selection of the appropriate clock source for system output. Its operation relies on two key pieces of information: the health status of each input clock (as indicated by Layer 1) and the user's clock selection preference.

The decision-making policy is as follows:

- **User Preference:** The system gives priority to the clock preferred by the user, as long as it is functioning correctly.
- **Automatic Switchover:** If the preferred clock fails but the alternative is still active, the system immediately designates the healthy clock as the desired source to maintain output continuity.
- **Safe Output State:** If both input clocks are inactive, Layer 2 will, in most cases, prevent either clock from being selected, allowing the output to transition to a defined safe state as governed by Layer 3. However, in rare situations—such as during critical timing events, protocol restrictions, or asynchronous transitions—the system may not be able to enforce the safe state immediately, but will do so as soon as operational conditions permit.

Throughout normal operation and during faults, Layer 2

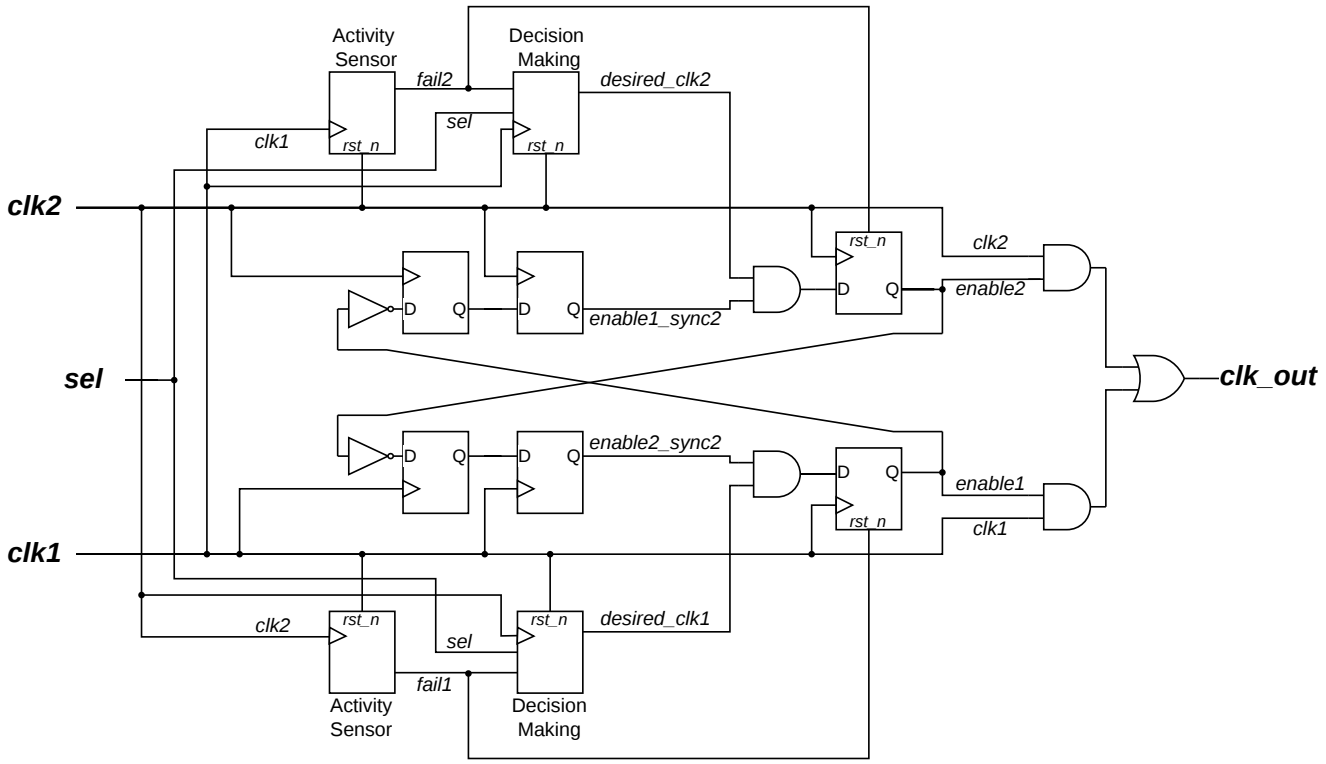

Fig. 5. Schematic of layered clock failover system, showing Activity Sensor, Decision Making, and Safe Handover logic for each clock domain. Mutual exclusion, synchronization, and safe output enforcement are all handled in Layer 3.

continually determines the appropriate source to use, producing "desired clock" signals that encode the system's clock selection intent at every instant. These signals serve as guidance for Layer 3, which implements glitch-free handover and output gating.

This policy enables rapid and robust responses to clock source failures, minimizing disruption and supporting scenarios ranging from straightforward preference enforcement to complex recovery situations. Verification of these outcomes, including handling of the safe output state, is presented in the subsequent results section.

D. Layer 3: Safe Handover

Layer 3 is responsible for safely implementing the clock selection determined by the Decision Making layer. Its core objective is to guarantee glitch-free and mutually exclusive switching between the two clock sources which ensure that only one source is ever enabled at a time, and that transitions do not cause spurious output or timing hazards.

- **Enable Synchronization, Pipelining, and Output Gating:** As illustrated in the block diagram Fig. 4, the "desired clock" signals from the Decision Making block are passed through dedicated flip-flop stages within each respective clock domain. These synchronizers play several roles: they *safely align enables to each clock domain*, *facilitate pipelining for improved timing and reduced critical path*, and *ensure enables change only on valid, stable clock edges*. This architectural principle avoids hazards and glitches, commonly seen with direct combinational gating.
- **Mutual Exclusion and Break-Before-Make:** The mutual exclusion logic, also outlined at a high level in Fig. 4, ensures that both output enables cannot ever be active simultaneously. This guaranties what is called break-before-make switching, which ensures clean, monotonic transitions at the output without the risk of simultaneous or glitchy clock drives. The detailed circuit implementation of this logic is further depicted in the schematic diagram in Fig. 5.
- **Safe Output State:** The principle of holding `clk_out` in a safe, inactive state (typically logic low) whenever no valid clock is available, such as when both inputs are inactive, reset is asserted, or synchronization dictates a temporary pause, which is incorporated at the top level in the block diagram (Fig. 4) and realized concretely in the output gating elements shown in the detailed schematic (Fig. 5).

In summary, the architecture described above establishes a robust and flexible framework for safe clock switching, combining enable synchronization, mutual exclusion, and well-defined safe-state handling. The high-level structure

Table I: Feature and implementation comparison for break-before-make, timer-based, and proposed GFCM architectures.

Design	Glitch-Free	Deadlock-Free	Dual-Stop Safety	Latency Bound	ALMs	LABs	ALUTs	DLRs	Max Freq. (MHz)
Break-before-make	Yes	No	No	Constant	3	2	4	6	580.05
Timer-based[1]	Yes	Yes	No	Freq.-dependent	12	4	5	10	367.92
Proposed GFCM	Yes	Yes	Yes	Constant	14	3	24	22	515.46

of the system is captured in the block diagram (Fig. 4), which illustrates the interplay between clock activity sensing, decision making, and output gating.

To provide deeper insight into the practical realization of these mechanisms, the complete logic is detailed in the accompanying schematic diagram (Fig. 5). This schematic reveals the precise signal paths, flip-flop stages, and gating elements that implement each architectural function, clarifying how abstract principles are mapped to concrete hardware. Readers are encouraged to refer to this diagram for a comprehensive understanding of the internal circuitry and to see how each conceptual block is instantiated in the final design.

IV. Results and Evaluation

A. Simulation and Functional Verification

To validate the correctness and reliability of the proposed design, the system was simulated under a range of critical scenarios. These test cases thoroughly exercise both normal operation and various fault conditions, including clock failure, recovery, and dual-stop situations.

Test case 1: Selected clock fails, alternate clock remains active (with full recovery even after both clocks fail)
When the selected clock (`clk2`) ceases toggling while the selector (`sel`) remains unchanged, but the other clock (`clk1`) remains active, the output (`clk_out`) seamlessly switches to and tracks the remaining live clock. **Remarkably, even if both clocks become inactive and `clk_out` is forced to a safe state (logic Low), the system will immediately and automatically resume correct operation—tracking any clock source that becomes active again, without requiring any selector update.** This robust functional recovery is demonstrated by the waveform in Fig. 6; specifically, the **leftmost circled region** highlights the output's recovery immediately after both clocks become inactive and then one resumes operation.

Test case 2: Selected clock recovers after failure
Demonstrating system flexibility, if the previously selected clock (`clk1`) resumes toggling after being stuck, the output (`clk_out`) returns to following this clock as soon as it recovers, as seen in the latter portion of Fig. 6.

Test case 3: Selected clock fails before subsequently both clocks stop
If both clock sources become inactive and the selector (`sel`) is toggled, the system moves `clk_out` to a safe state (logic Low), preventing invalid transitions or glitches. This scenario is depicted in Fig. 7.

The simulation waveforms verify that the proposed architecture reliably supports glitch-free and fully autonomous failover across all tested scenarios, outperforming conventional schemes in both flexibility and safety.

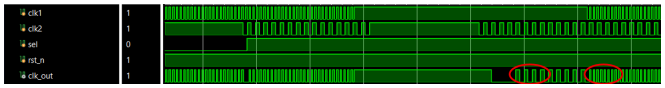

Fig. 6. Simulation results (cases 1 & 2): **Left**—Output switches smoothly to the live alternate clock if the selected clock fails; **Right**—Output resumes following a recovered clock after temporary failure.

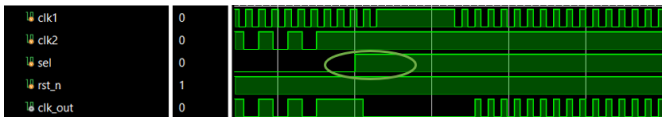

Fig. 7. Simulation result (case 3): When both clock sources are inactive and the selector switches, the output is safely forced to logic Low, maintaining robust fault tolerance.

B. FPGA Implementation and Resource Utilization

All architectures were designed in Verilog HDL and synthesized for the Intel Cyclone V 5CEFA2F23C8 FPGA. The measured resource utilization and timing performance for each design are summarized in Table I, enabling a direct side-by-side comparison under consistent tool and device conditions.

Table I reveals that the break-before-make (BBM) design achieves the lowest area consumption, requiring only 3 ALMs, 2 LABs, 4 ALUTs, and 6 DLRs, and reaches the highest maximum frequency at 580.05 MHz. However, this minimalism comes at the price of compromised functionality, as the BBM design lacks both deadlock-free operation and dual-stop safety. The timer-based architecture improves functional reliability with deadlock-free switching, but still fails to provide dual-stop safety. This approach sees a moderate increase in resource usage (12 ALMs, 4 LABs, 5 ALUTs, 10 DLRs) and attains a reduced maximum frequency of 367.92 MHz due to frequency-dependent timing overhead.

By contrast, the proposed GFCM architecture delivers an unique combination of full glitch-free behavior, deadlock avoidance, and dual-stop safety—all with constant-latency operation. While GFCM exhibits a moderate area increase (14 ALMs, 3 LABs, 24 ALUTs, 22 DLRs) relative to BBM, it achieves a high maximum operating frequency of 515.46 MHz—substantially higher than the timer-based scheme and only slightly below the resource-minimal BBM design. This demonstrates an advantageous trade-off: a mod-

est increase in hardware resources enables not only high-speed operation, but also robust, autonomous fault recovery and safe performance even under extreme failure scenarios, which cannot be guaranteed by the other two designs. Thus, the proposed GFCM is exceptionally suited for demanding applications that prioritize both performance and functional safety.

V. Conclusion

This work has presented a robust, glitch-free clock multiplexer architecture capable of fully autonomous and safe switchover between asynchronous clock sources. By integrating the key strengths of traditional combinational, break-before-make, and timer-based schemes, the proposed GFCM solution achieves comprehensive coverage of failure scenarios, ensuring glitch immunity, deadlock freedom, and immediate recovery without manual intervention.

Functional verification confirmed the system's ability to sustain seamless operation and failover across a broad range of test conditions, while FPGA implementation results on the Intel Cyclone V 5CEFA2F23C8 device demonstrated that these advanced safety features are achieved with modest resource overhead and competitive maximum frequency. Comparative analysis highlighted superior performance and resilience over established approaches, making the proposed architecture well-suited for safety-critical and high-availability digital systems.

Future work may extend this framework to support multiplexers with more than two clock domains, or further optimize the architecture for ultra-low-power or high-performance applications. Additional directions include adaptive thresholding for failure detection by exploiting input clock frequency ratios, as well as enhancing monitoring efficiency through resource-optimized or pseudo-randomized counter designs.

References

[1] S. Zeidler, O. Schrape, A. Breitenreiter, and M. Krstić, "A glitch-free clock multiplexer for non-continuously running clocks," in *2020 23rd Euromicro Conference on Digital System Design (DSD)*, 2020, pp. 11–15.